DATA STRUCTURES AND ALGORITHMS

LECTURE 8

Lect. PhD. Oneț-Marian Zsuzsanna

Babeș - Bolyai University
Computer Science and Mathematics Faculty

2025 - 2026

In Lecture 7...

- Doubly linked list on array
- Stack - representation options

Today

- Queue - representation options
- Dequeue
- Priority Queue - representation options
- Binary heap

ADT Queue - Recap

- The ADT Queue represents a container in which access to the elements is restricted to the two ends of the container, called *front* and *rear*.
  - When a new element is added (pushed), it has to be added to the *rear* of the queue.
  - When an element is removed (popped), it will be the one at the *front* of the queue.
- Because of this restricted access, the queue is said to have a **FIFO** policy: First In First Out.

Queue - Representation

- What data structures can be used to implement a Queue?
  - Dynamic Array - circular array (already discussed)
  - Singly Linked List
  - Doubly Linked List

Queue - representation on a SLL

- If we want to implement a Queue using a singly linked list, where should we place the *front* and the *rear* of the queue?

Queue - representation on a SLL

- If we want to implement a Queue using a singly linked list, where should we place the *front* and the *rear* of the queue?
- In theory, we have two options:
  - Put *front* at the beginning of the list and *rear* at the end
  - Put *front* at the end of the list and *rear* at the beginning
- In either case we will have one operation (push or pop) that will have $\Theta(n)$ complexity.

Queue - representation on a SLL

- We can improve the complexity of the operations if, even though the list is singly linked, we keep both the head and the tail of the list.
- What should the tail of the list be: the *front* or the *rear* of the queue?

Queue - representation on a SLL

- We can improve the complexity of the operations if, even though the list is singly linked, we keep both the head and the tail of the list.
- What should the tail of the list be: the *front* or the *rear* of the queue?
- We can easily insert after the tail in a SLL, but we cannot remove it in $\Theta(1)$ time (you need the previous node for removal).

Queue - representation on a DLL

- If we want to implement a Queue using a doubly linked list, where should we place the *front* and the *rear* of the queue?

Queue - representation on a DLL

- If we want to implement a Queue using a doubly linked list, where should we place the *front* and the *rear* of the queue?
- In theory, we have two options:
  - Put *front* at the beginning of the list and *rear* at the end
  - Put *front* at the end of the list and *rear* at the beginning

ADT Deque

- The ADT Deque (Double Ended Queue) is a container in which we can insert and delete from both ends:
  - We have *push_front* and *push_back*
  - We have *pop_front* and *pop_back*
  - We have *top_front* and *top_back*
- We can simulate both stacks and queues with a deque if we restrict ourselves to using only part of the operations.

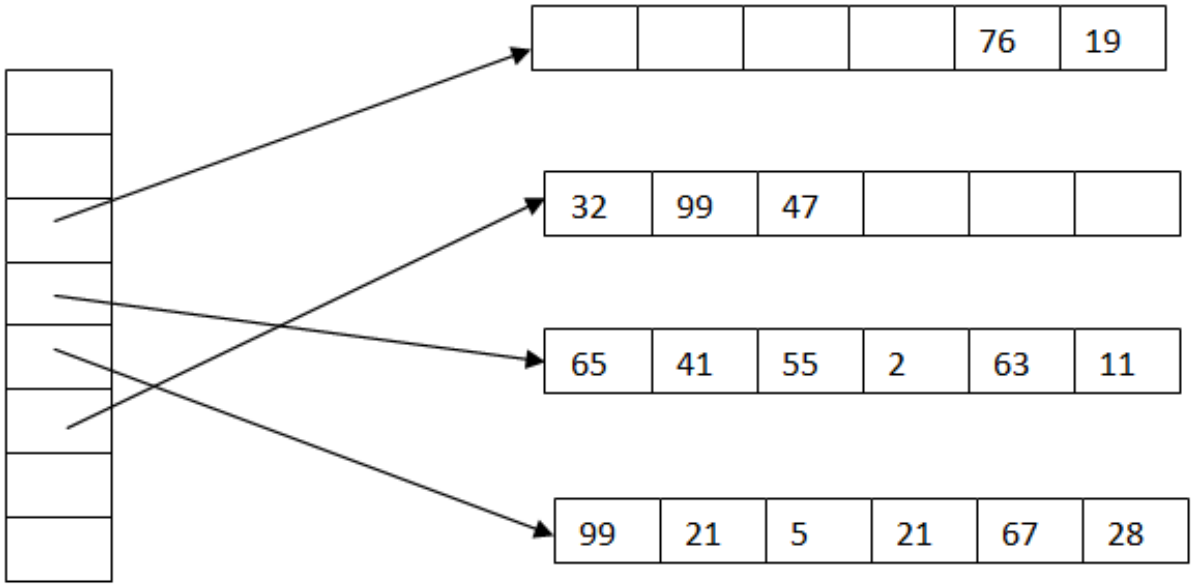
ADT Deque

- Possible (good) representations for a Deque:
  - Circular Array
  - Doubly Linked List
  - A dynamic array of constant size arrays

ADT Deque - Representation

- An interesting representation for a deque is to use a dynamic array of fixed size arrays:
  - Place the elements in fixed size arrays (blocks).
  - Keep a dynamic array with the addresses of these blocks.
  - Every block is full, except for the first and last ones.
  - The first block is filled from right to left.
  - The last block is filled from left to right.
  - If the first or last block is full, a new one is created and its address is put in the dynamic array.
  - If the dynamic array is full, a larger one is allocated, and the addresses of the blocks are copied (but elements are not moved).

Deque - Example



- Elements of the deque: 76, 19, 65, ..., 11, 99, ..., 28, 32, 99, 47

Deque - Example

- Information (fields) we need to represent a deque using a dynamic array of blocks:
  - Block size
  - The dynamic array with the addresses of the blocks
  - Capacity of the dynamic array
  - First occupied position in the dynamic array
  - First occupied position in the first block
  - Last occupied position in the dynamic array
  - Last occupied position in the last block
  - Optionally, we can keep count of the total number of elements in the deque as well.

ADT Deque

- The above representation is used by C++, because in C++ deques have another important operation besides the already mentioned ones: access to element based on position.
- What is the complexity of this operation for this representation?
- And on the alternative representations?

ADT Priority Queue - Recap

- The ADT Priority Queue is a container in which each element has an associated *priority* (of type *TPriority*).
- In a Priority Queue access to the elements is restricted: we can access only the element with the highest priority.
- Because of this restricted access, we say that the Priority Queue works based on a **HPF - Highest Priority First** policy.

Priority Queue - Representation

- What data structures can be used to implement a priority queue?
  - Dynamic Array
  - Linked List
  - (Binary) Heap - will be discussed later

Priority Queue - Representation

- If the representation is a Dynamic Array or a Linked List we have to decide how we store the elements in the array/list:
  - we can keep the elements ordered by their priorities
    - Where would you put the element with the highest priority?
  - we can keep the elements in the order in which they were inserted

Priority Queue - Representation

- Complexity of the main operations for the two representation options:

Operation	**Sorted**	**Non-sorted**
push	$O(n)$	$\Theta(1)$
pop	$\Theta(1)$	$\Theta(n)$
top	$\Theta(1)$	$\Theta(n)$

- What happens if we keep in a separate field the element with the highest priority?

Binary Heap

- A binary heap is a data structure that can be used as an efficient representation for Priority Queues.
- A binary heap is a kind of hybrid between a dynamic array and a binary tree.
- The elements of the heap are actually stored in the dynamic array, but the array is visualized as a binary tree.

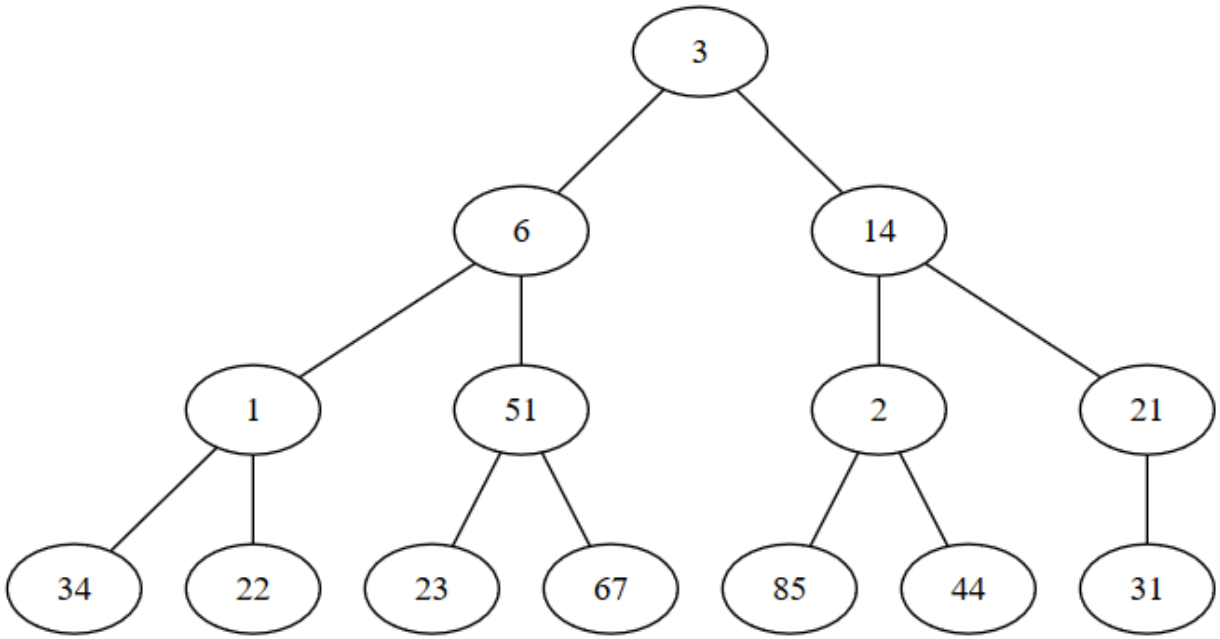
Binary Heap

- Assume that we have the following array (upper row contains positions, lower row contains elements):

1	2	3	4	5	6	7	8	9	10	11	12	13	14
3	6	14	1	51	2	21	34	22	23	67	85	44	31

Binary Heap

- We can visualize this array as a binary tree, where the root is the first element of the array, its children are the next two elements, and so on. Each node has exactly 2 children, except for the last two rows, but there the children of the nodes are completed from left to right.



Binary Heap

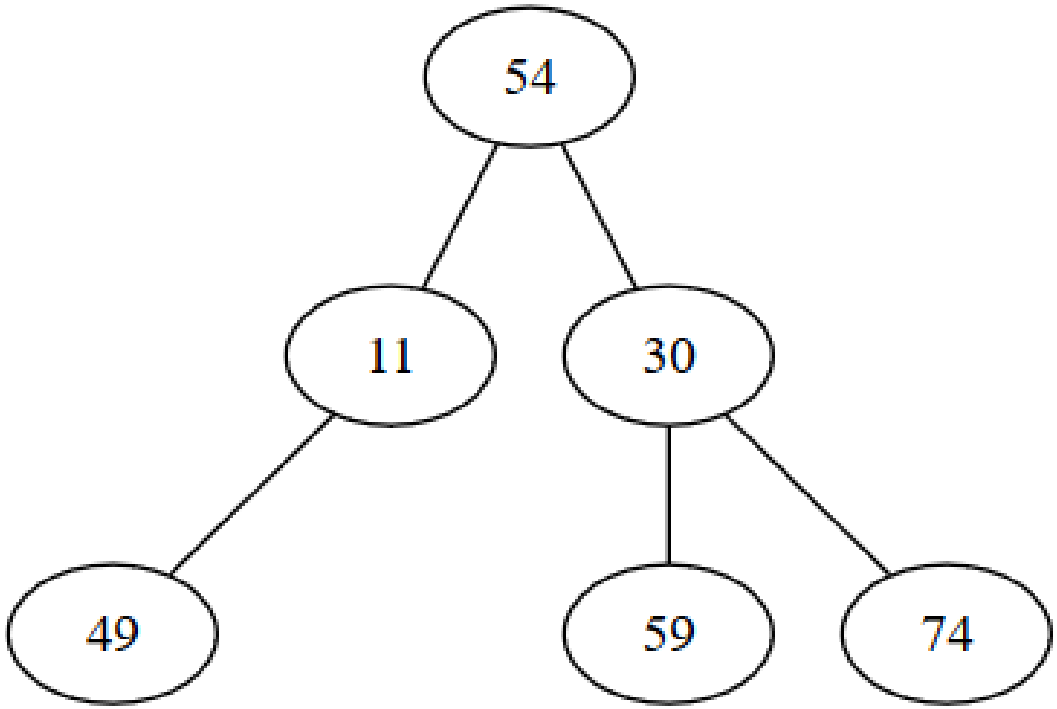
- If the elements of the array are: $a_1, a_2, a_3, ..., a_n$, we know that:
  - a_1 is the root of the heap
  - for an element from position i, its children are on positions $2 * i$ and $2 * i + 1$ (if $2 * i$ and $2 * i + 1$ is less than or equal to n)
  - for an element from position i (i > 1), the parent of the element is on position $[i/2]$ (integer part of i/2)

Binary Heap

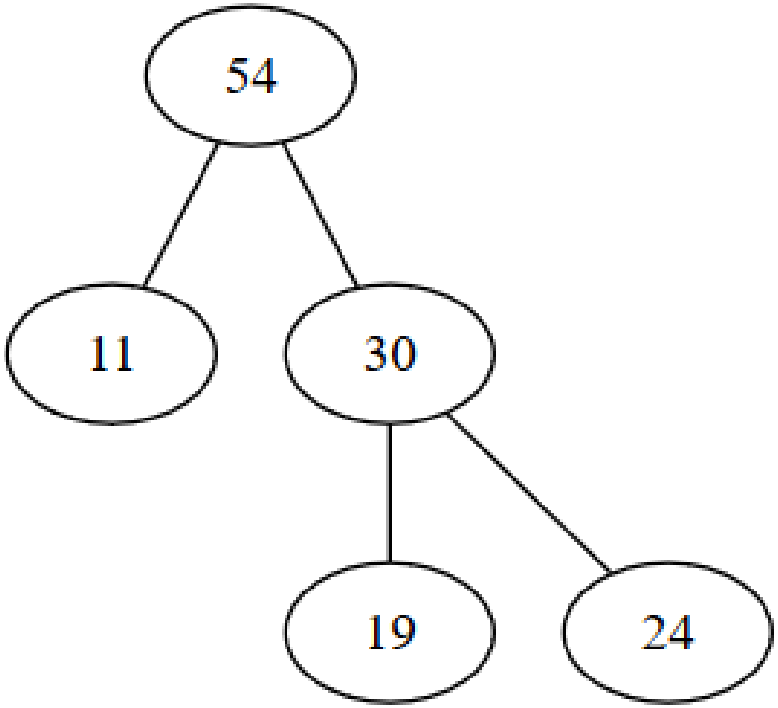
- A *binary heap* is an array that can be visualized as a binary tree having a *heap structure* and a *heap property*.
  - *Heap structure:* in the binary tree every node has exactly 2 children, except for the last two levels, where children are completed from left to right.
  - *Heap property:* $a_i \geq a_{2*i}$ (if $2 * i \leq n$) and $a_i \geq a_{2*i+1}$ (if $2 * i + 1 \leq n$)
  - The $\geq$ relation between a node and both its descendants can be generalized (other relations can be used as well).

Binary Heap - Examples I

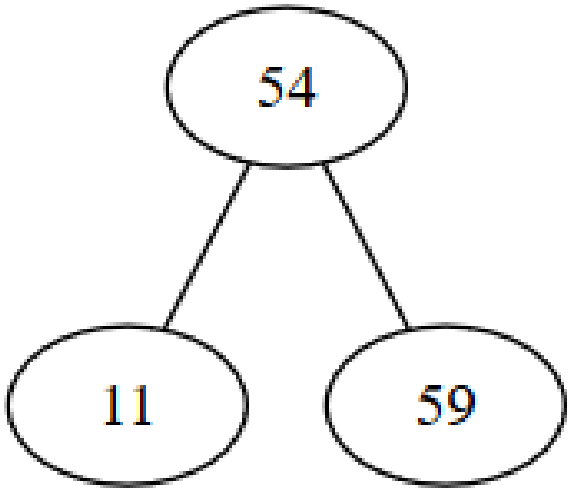
- Are the following binary trees heaps? If yes, specify the relation between a node and its children. If not, specify if the problem is with the structure, the property, or both.



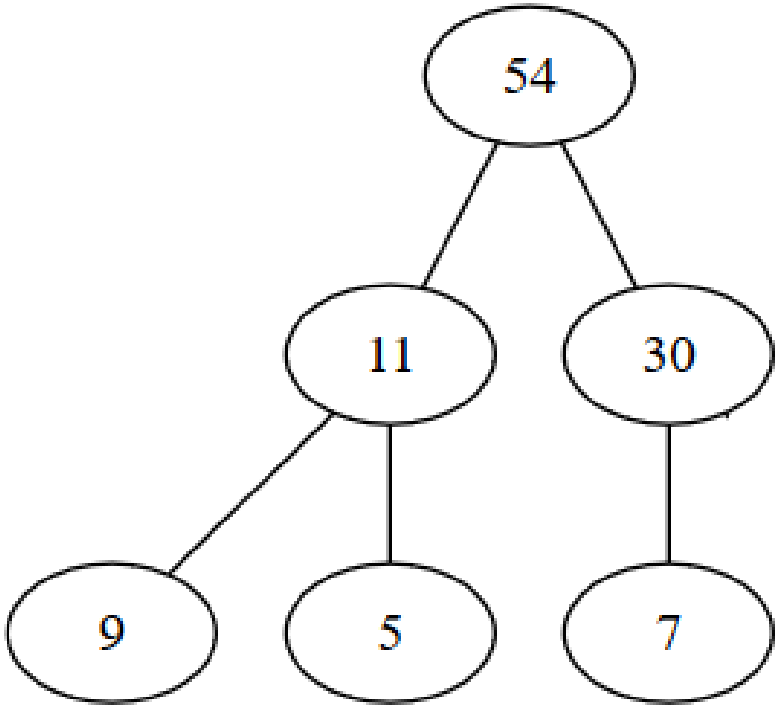
Binary Heap - Examples II



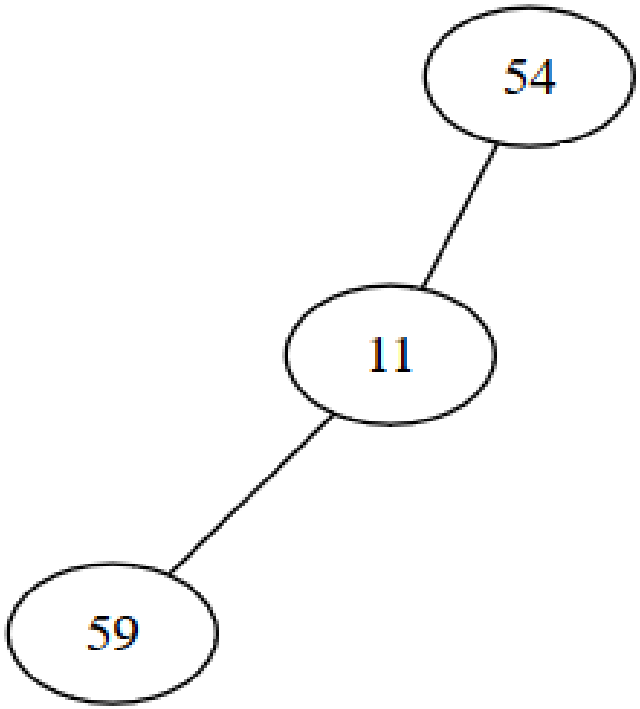
Binary Heap - Examples III



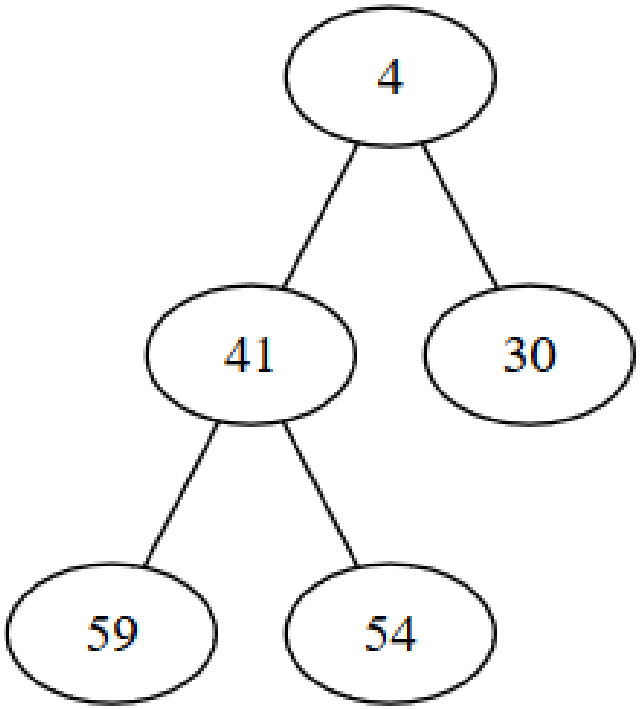
Binary Heap - Examples IV



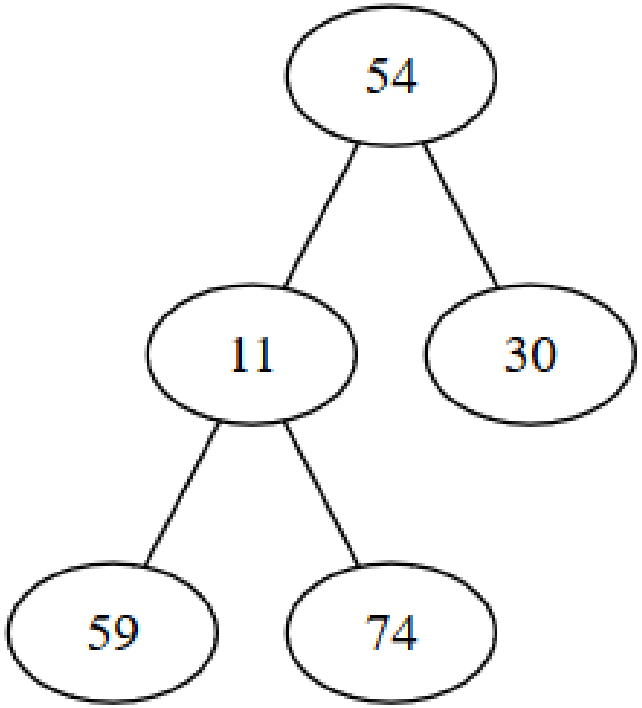
Binary Heap - Examples V



Binary Heap - Examples VI



Binary Heap - Examples VII



Binary Heap - Examples VIII

- Are the following arrays valid heaps? If not, transform them into a valid heap by swapping two elements.
  1. [70, 10, 50, 7, 1, 33, 3, 8]
  2. [1, 2, 4, 8, 16, 32, 64, 65, 10]
  3. [10, 12, 100, 60, 13, 102, 101, 80, 90, 14, 15, 16]

Binary Heap - Notes

- If we use the $\geq$ relation, we will have a *MAX-HEAP*. Do you know why?

Binary Heap - Notes

- If we use the $\geq$ relation, we will have a *MAX-HEAP*. Do you know why?
- If we use the $\leq$ relation, we will have a *MIN-HEAP*. Do you know why?

Binary Heap - Notes

- If we use the $\geq$ relation, we will have a *MAX-HEAP*. Do you know why?
- If we use the $\leq$ relation, we will have a *MIN-HEAP*. Do you know why?
- The height of a heap with n elements is $log_2 n$.

Binary Heap - operations

- A heap can be used as representation for a Priority Queue and it has two specific operations:
  - add a new element in the heap (in such a way that we keep both the heap structure and the heap property).
  - remove (we always remove the root of the heap - no other element can be removed).

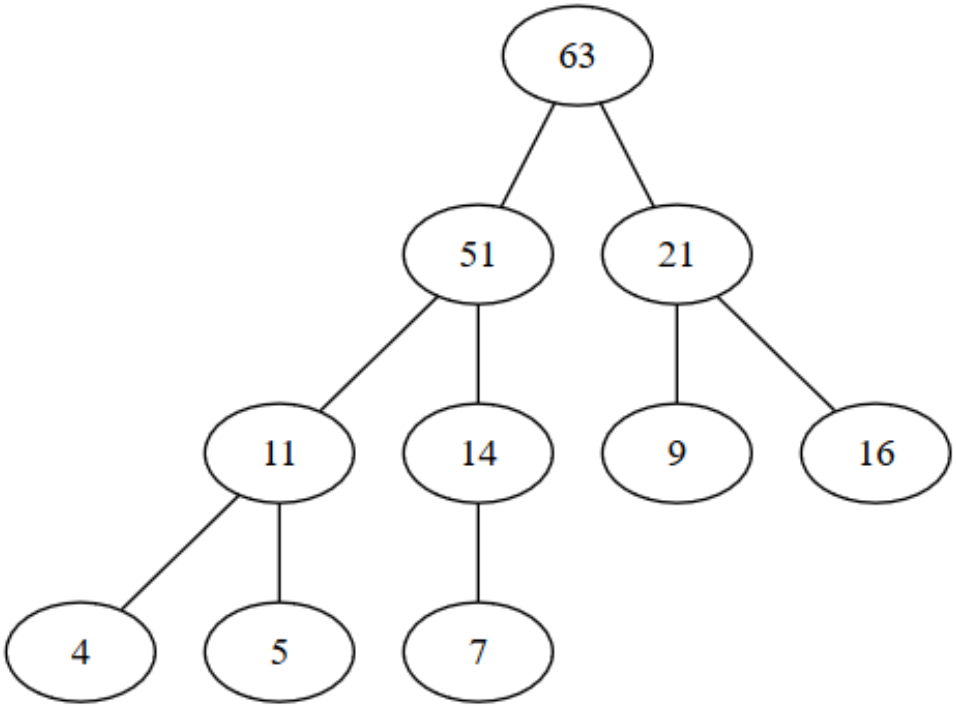
Binary Heap - representation

Heap:
- cap: Integer
- len: Integer
- elems: TElem[]

- Depending on the problem, you might need to have a *relation* as well as part of the heap.
- For the implementation we will assume that we have a MAX-HEAP.

Binary Heap - Add - example

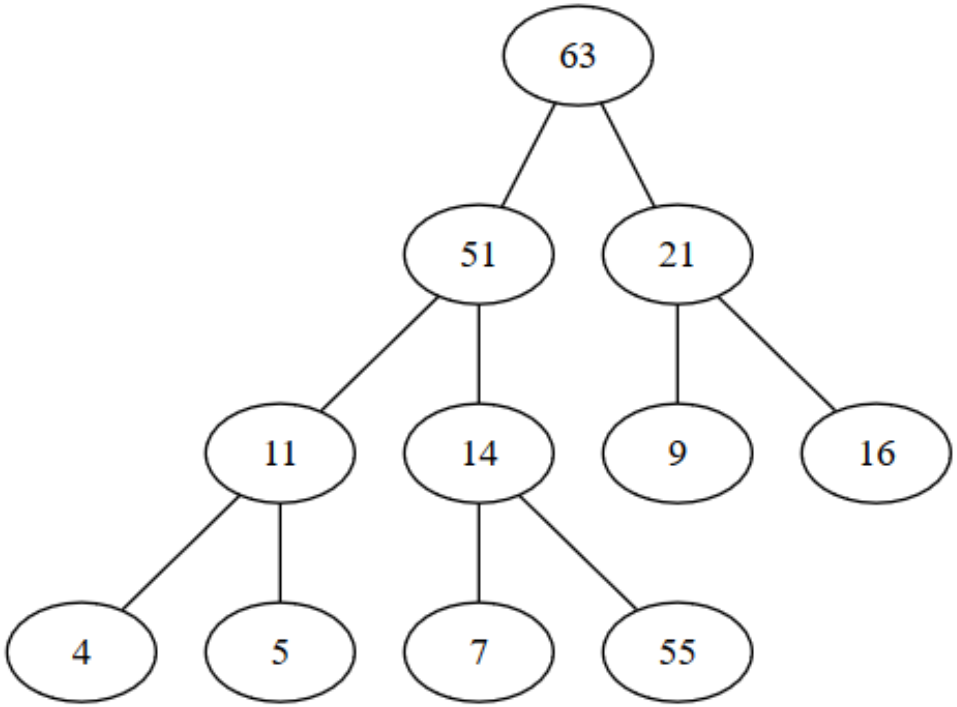
- Consider the following (MAX) heap:



- Let's add the number 55 to the heap.

Binary Heap - Add - example

- In order to keep the *heap structure*, we will add the new node as the right child of the node 14 (and as the last element of the array in which the elements are kept).

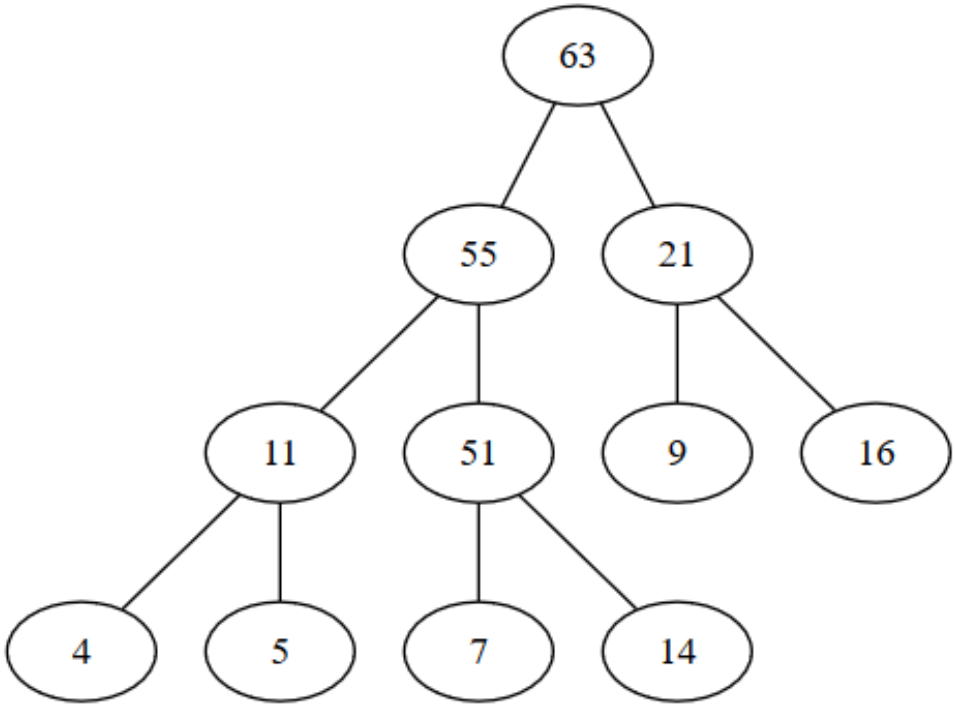


Binary Heap - Add - example

- *Heap property* is not kept: 14 has as child node 55 (since it is a MAX-heap, each node has to be greater than or equal to its descendants).
- In order to restore the heap property, we will start a *bubble-up* process: we will keep swapping the value of the new node with the value of its parent node, until it gets to its final place. No other node from the heap is changed.

Binary Heap - Add - example

- When *bubble-up* ends:



Binary Heap - add

```
subalgorithm add(heap, e) is:
//heap - a heap
//e - the element to be added
    if heap.len = heap.cap then
        @ resize
    end-if
    heap.elems[heap.len+1] ← e
    heap.len ← heap.len + 1
    bubble-up(heap, heap.len)
end-subalgorithm
```

Binary Heap - add

```
subalgorithm bubble-up (heap, p) is:
//heap - a heap
//p - position from which we bubble the new node up
    poz ← p
    elem ← heap.elems[p]
    parent ← p / 2
    while poz > 1 and elem > heap.elems[parent] execute
        //move parent down
        heap.elems[poz] ← heap.elems[parent]
        poz ← parent
        parent ← poz / 2
    end-while
    heap.elems[poz] ← elem
end-subalgorithm
```

- Complexity:

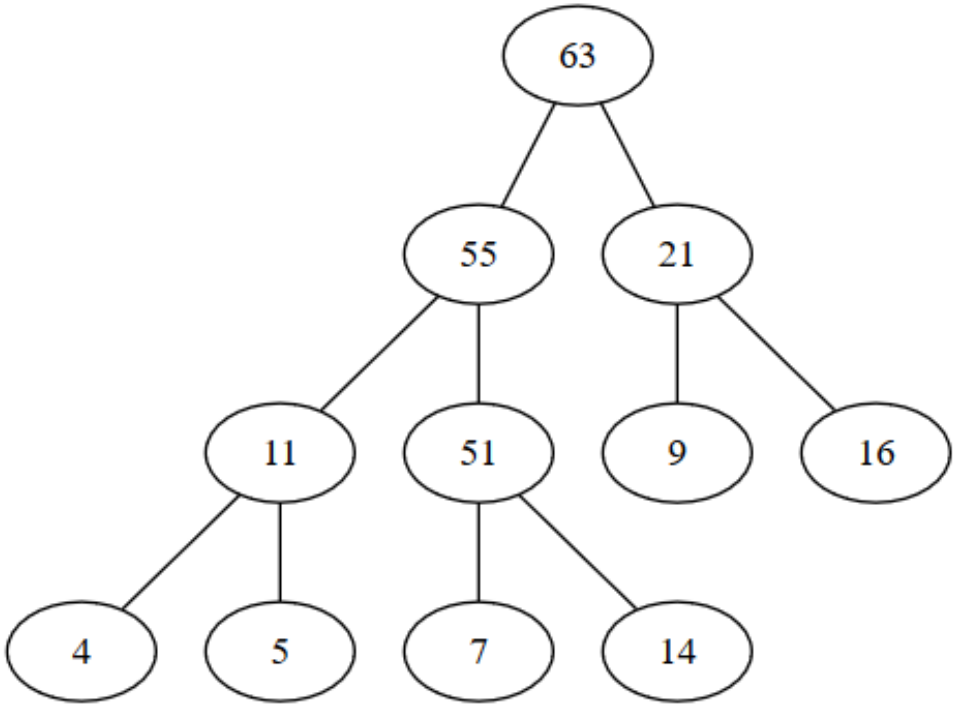
Binary Heap - add

```
subalgorithm bubble-up (heap, p) is:
//heap - a heap
//p - position from which we bubble the new node up
    poz ← p
    elem ← heap.elems[p]
    parent ← p / 2
    while poz > 1 and elem > heap.elems[parent] execute
        //move parent down
        heap.elems[poz] ← heap.elems[parent]
        poz ← parent
        parent ← poz / 2
    end-while
    heap.elems[poz] ← elem
end-subalgorithm
```

- Complexity: $O(log_2 n)$
- Can you give an example when the complexity of the algorithm is less than $log_2 n$ (best case scenario)?

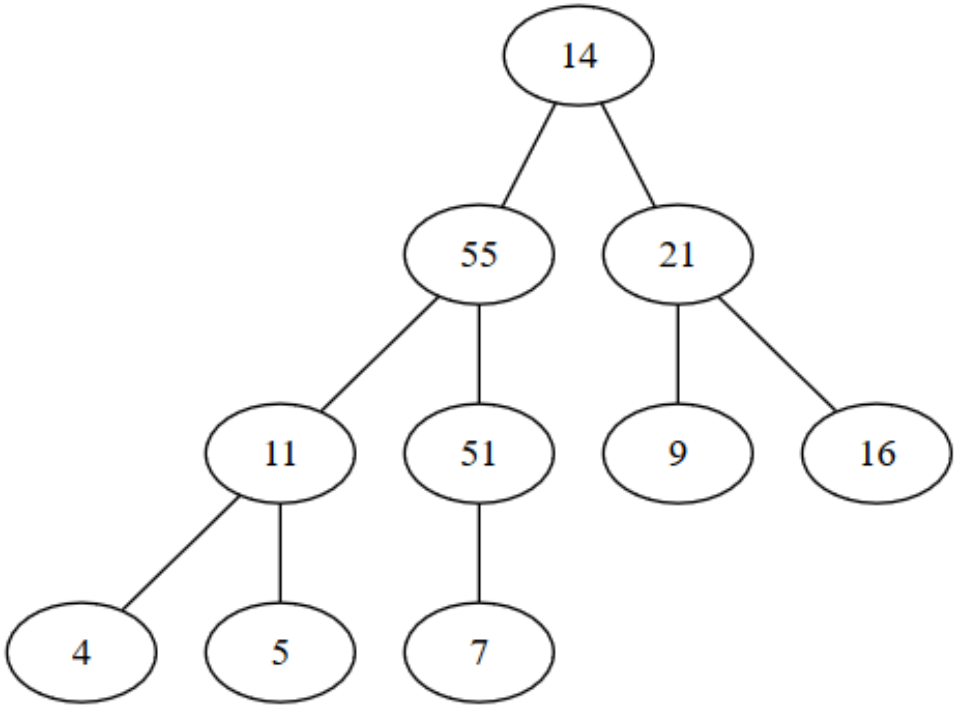
Binary Heap - Remove - example

- From a heap we can only remove the root element.



Binary Heap - Remove - example

- In order to keep the *heap structure*, when we remove the root, we are going to move the last element from the array to be the root.

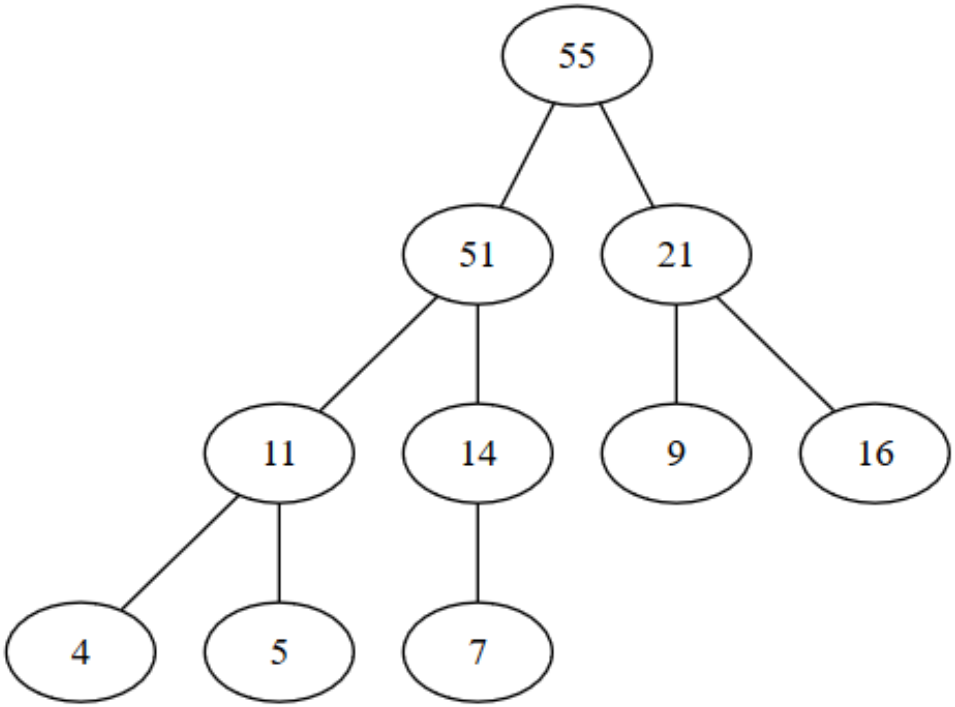


Binary Heap - Remove - example

- *Heap property* is not kept: the root is no longer the maximum element.
- In order to restore the heap property, we will start a *bubble-down* process, where the new node will be swapped with its maximum child, until it becomes a leaf, or until it will be greater than both children.

Binary Heap - Remove - example

- When the bubble-down process ends:



Binary Heap - remove

```
function remove(heap) is:
//heap - is a heap
    if heap.len = 0 then
        @ error - empty heap
     end-if
    deletedElem ← heap.elems[1]
    heap.elems[1] ← heap.elems[heap.len]
    heap.len ← heap.len - 1
    bubble-down(heap, 1)
    remove ← deletedElem
end-function
```

Binary Heap - remove

```
subalgorithm bubble-down(heap, p) is:
//heap - is a heap
//p - position from which we move down the element
    poz ← p
    elem ← heap.elems[p]
    while poz < heap.len execute
        maxChild ← -1
        if poz * 2 ≤ heap.len then
        //it has a left child, assume it is the maximum
            maxChild ← poz*2
        end-if
        if poz*2+1 ≤ heap.len and heap.elems[2*poz+1]>heap.elems[2*poz] th
        //it has two children and the right is greater
            maxChild ← poz*2 + 1
        end-if
//continued on the next slide...
```

Binary Heap - remove

```
    if maxChild ≠ -1 and heap.elems[maxChild] > elem then
        tmp ← heap.elems[poz]
        heap.elems[poz] ← heap.elems[maxChild]
        heap.elems[maxChild] ← tmp
        poz ← maxChild
    else
        poz ← heap.len + 1
        //to stop the while loop
    end-if
  end-while
end-subalgorithm
```

- Complexity:

Binary Heap - remove

```
    if maxChild ≠ -1 and heap.elems[maxChild] > elem then
        tmp ← heap.elems[poz]
        heap.elems[poz] ← heap.elems[maxChild]
        heap.elems[maxChild] ← tmp
        poz ← maxChild
    else
        poz ← heap.len + 1
        //to stop the while loop
    end-if
  end-while
end-subalgorithm
```

- Complexity: $O(log_2 n)$
- Can you give an example when the complexity of the algorithm is less than $log_2 n$ (best case scenario)?

Questions

- In a max-heap where can we find the:
  - maximum element of the array?

Questions

- In a max-heap where can we find the:
  - maximum element of the array?
  - minimum element of the array?

Questions

- In a max-heap where can we find the:
  - maximum element of the array?
  - minimum element of the array?
- Assume you have a MAX-HEAP and you need to add an operation that returns the minimum element of the heap. How would you implements this operation, using constant time and space? (Note: we only want to return the minimum, we do not want to be able to remove it).

Exercises

- Consider an initially empty Binary MAX-HEAP and insert the elements 8, 27, 13, 15*, 32, 20, 12, 50*, 29, 11* in it. Draw the heap in the tree form after the insertion of the elements marked with a * (3 drawings). Remove 3 elements from the heap and draw the tree form after every removal (3 drawings).

- Insert the following elements, in this order, into an initially empty MIN-HEAP: 15, 17, 9, 11, 5, 19, 7. Remove all the elements, one by one, in order from the resulting MIN HEAP. Draw the heap after every second operation (after adding 17, 11, 19, etc.)

Heap-sort

- There is a sorting algorithm, called *Heap-sort*, that is based on the use of a heap.
- In the following we are going to assume that we want to sort a sequence in ascending order.
- Let's sort the following sequence: [6, 1, 3, 9, 11, 4, 2, 5]

Heap-sort - Naive approach

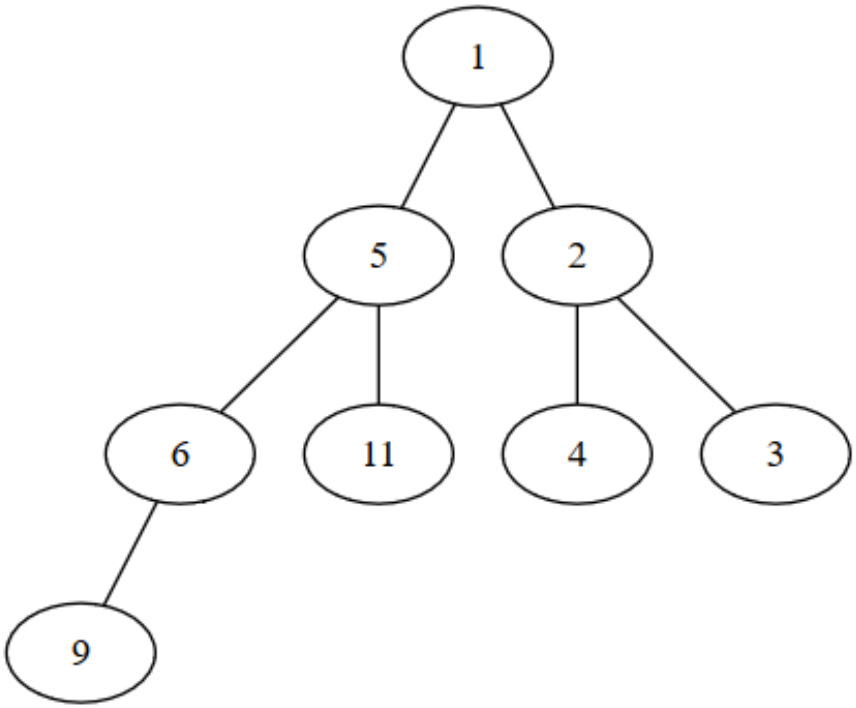
- Based on what we know so far, we can guess how heap-sort works:

Heap-sort - Naive approach

- Based on what we know so far, we can guess how heap-sort works:
  - Build a min-heap adding elements one-by-one to it.
  - Start removing elements from the min-heap: they will be removed in the sorted order.

Heap-sort - Naive approach

- The heap when all the elements were added:



- When we remove the elements one-by-one we will have: 1, 2, 3, 4, 5, 6, 9, 11.

Heap-sort - Naive approach

- What is the time complexity of the heap-sort algorithm described above?

Heap-sort - Naive approach

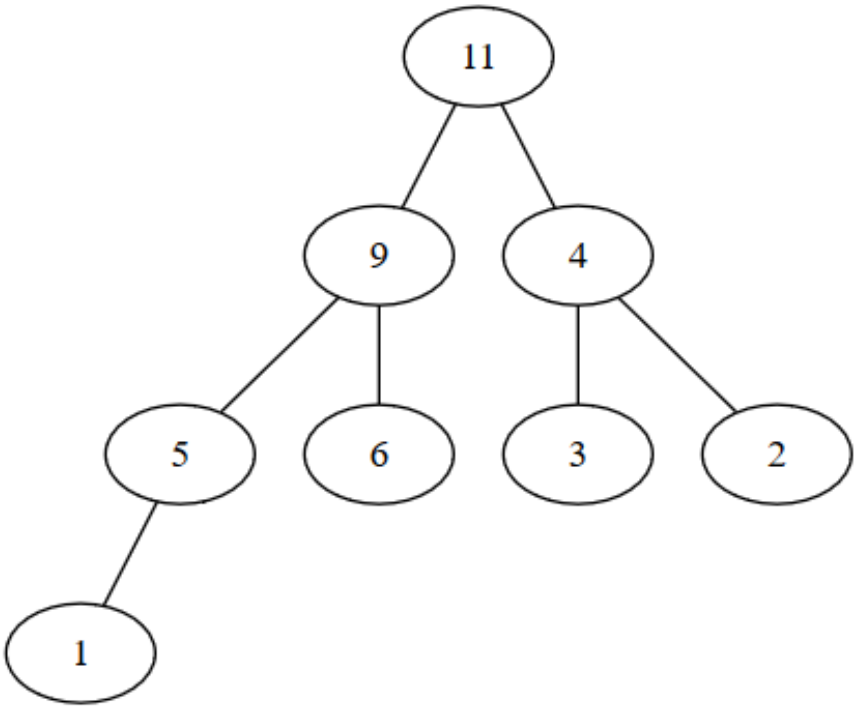
- What is the time complexity of the heap-sort algorithm described above?
- The time complexity of the algorithm is $O(nlog_2 n)$
- What is the extra space complexity of the heap-sort algorithm described above (do we need an extra array)?

Heap-sort - Naive approach

- What is the time complexity of the heap-sort algorithm described above?
- The time complexity of the algorithm is $O(nlog_2 n)$
- What is the extra space complexity of the heap-sort algorithm described above (do we need an extra array)?
- The extra space complexity of the algorithm is $\Theta(n)$ - we need an extra array.

Heap-sort - Better approach

- If instead of building a min-heap, we build a max-heap (even if we want to do ascending sorting), we do not need the extra array.



Heap-sort - Better approach

- We can improve the time complexity of building the heap as well.

Heap-sort - Better approach

- We can improve the time complexity of building the heap as well.
  - If we have an unsorted array, we can transform it easier into a heap: the second half of the array will contain leaves, they can be left where they are.
  - Starting from the first non-leaf element (and going towards the beginning of the array), we will just call *bubble-down* for every element.
  - Time complexity of this approach: $O(n)$ (but removing the elements from the heap is still $O(nlog_2n)$)

Priority Queue - Representation on a binary heap

- When an element is pushed to the priority queue, it is simply added to the heap (and bubbled-up if needed)
- When an element is popped from the priority queue, the root is removed from the heap (and bubble-down is performed if needed)
- Top simply returns the root of the heap.

Priority Queue - Representation

- Let's complete our table with the complexity of the operations if we use a heap as representation:

Operation	**Sorted**	**Non-sorted**	**Heap**
push	O(n)	$\Theta(1)$	$O(log_2 n)$
pop	$\Theta(1)$	$\Theta(n)$	$O(log_2 n)$
top	$\Theta(1)$	$\Theta(n)$	$\Theta(1)$

Priority Queue - Representation

- Let's complete our table with the complexity of the operations if we use a heap as representation:

Operation	**Sorted**	**Non-sorted**	**Heap**
push	O(n)	$\Theta(1)$	$O(log_2 n)$
pop	$\Theta(1)$	$\Theta(n)$	$O(log_2 n)$
top	$\Theta(1)$	$\Theta(n)$	$\Theta(1)$

- Consider the total complexity of the following sequence of operations:
  - start with an empty priority queue
  - push n random elements to the priority queue
  - perform pop n times

Priority Queue - Extension

- We have discussed the *standard* interface of a Priority Queue, the one that contains the following operations:
  - push
  - pop
  - top
  - isEmpty
  - init
- Sometimes, depending on the problem to be solved, it can be useful to have the following three operations as well:
  - increase/decrease the priority of an existing element
  - delete an arbitrary element
  - merge two priority queues

Priority Queue - Extension

- What is the complexity of these three extra operations if we use as representation a binary heap?
  - Increasing/decreasing the priority of an existing element is $O(log_2 n)$ if we know the position where the element is.
  - Deleting an arbitrary element is $O(log_2 n)$ if we know the position where the element is.
  - Merging two priority queues has complexity $\Theta(n + m)$ (assume the priority queues have n and m elements) - copy them in a single array and apply heapify on them.

Priority Queue - Other representations

- If we do not want to merge priority queues, a binary heap is a good representation. If we need the merge operation, there are other heap data structures that can be used, which offer a better complexity.
- Out of these data structures we are going to discuss one: the *binomial heap*.

Summary

- Today we have talked about:
  - Queue
  - Dequeue
  - Priority queue
  - Binary heap
- Extra reading: a nice problem statement